

Automated Digitization of Gas Chromatography–Mass Spectrometry Spectra from Vector PDFs Using Peak–Label Assignment Algorithms

^{1*} and ¹

^{1*}Combustion Gasification and Propulsion Laboratory, Indian Institute of Science, CV Raman Rd, Bengaluru, 560012, Karnataka, India.

*Corresponding author(s). E-mail(s): ;
Contributing authors: ;

Abstract

Gas Chromatography–Mass Spectrometry (GC-MS) instruments produce semantically dense scientific charts embedded as vector graphics in PDF reports. Manual extraction of peak labels and relative abundance values from these documents is time-consuming and error-prone. This paper investigates the logical structure recovery of GC-MS spectra from vector PDFs, addressing the challenge of re-linking disassociated primitives where textual labels and graphical peak paths are stored as independent objects. We frame the problem as a label-to-peak assignment task, and three assignment algorithms are developed and compared: a greedy nearest-neighbor approach, a dynamic programming method enforcing monotone matching, and a Hungarian bipartite matching algorithm with geometric peak validation. A common preprocessing pipeline extracts text and vector segments via PyMuPDF. Parallel implementations are provided in Python (multiprocessing), C++ with OpenMP, and C++ with MPI. Experiments are conducted on a dataset of 231 GC-MS PDF spectra spanning diverse layout variations and compound classes. The Hungarian and greedy methods attain the lowest RMSE on relative abundance (0.34–0.38), outperforming the DP approach (RMSE 1.79). C++ OpenMP and MPI implementations achieve approximately $4.5\times$ speedup at 6 threads over the serial baseline. Vector-based extraction eliminates OCR uncertainty and enables precise, scalable GC-MS digitization. The proposed pipeline provides a reproducible, open alternative to manual transcription, with practical utility for downstream cheminformatics workflows.

Keywords: GC-MS digitization, peak-label assignment, vector PDF extraction, dynamic programming, Hungarian algorithm, parallel computing

1 Introduction

Gas chromatography–mass spectrometry (GC-MS) is a widely used analytical technique across chemical, biological, pharmaceutical, and environmental sciences, enabling identification and

quantification of compounds within complex mixtures (1). A GC-MS instrument separates a sample into its constituent compounds by retention time and subsequently fragments each compound, producing a characteristic mass spectrum — a bar graph of ion mass-to-charge ratio (m/z) on the x -axis against relative abundance (RA) on

the y -axis, where each vertical bar, referred to as a *peak*, represents a detected ion fragment, and the *relative abundance* of each peak expresses its intensity as a percentage of the most intense peak in the spectrum. Despite their analytical importance, GC-MS instruments almost universally export final reports as PDF files containing vector-rendered graphics, and extracting the numerical data — peak m/z values and their relative abundances from these files has traditionally been a manual process in which an analyst visually inspects the spectrum and transcribes each value by hand, an approach that is both time-consuming and prone to human error, particularly for spectra containing large numbers of peaks or tightly spaced labels. Existing automated approaches fall into two broad categories, both of which carry significant limitations: raster-based methods render the PDF page as a bitmap image and apply Optical Character Recognition (OCR) to detect axis labels and peak annotations, but OCR introduces character recognition errors — digits such as 1 and 7 are frequently confused — and pixel-level axis detection is sensitive to rendering resolution and font size; vector heuristic methods exploit the fact that modern PDF files embed text and path elements as explicit objects, avoiding OCR entirely, but naive heuristic approaches that simply match the nearest text token to the nearest vertical segment fail when labels are grouped into a single text object (e.g., 131415 rendered as one token), when labels are offset from their corresponding peaks, or when the axis scale differs across documents. In this paper, we make the following contributions: (1) a common preprocessing pipeline that extracts text words and vector segments from GC-MS PDFs using PyMuPDF (2), detects axis geometry, and resolves grouped numeric labels through a scored partition algorithm; (2) three peak-label assignment algorithms of increasing sophistication — a *Greedy* nearest-neighbour approach, a *Dynamic Programming* (DP) approach enforcing monotone left-to-right assignment, and a *Hungarian matching* algorithm that finds the globally optimal assignment via minimum-cost bipartite matching with geometric validation; (3) parallel implementations of all three algorithms in Python (multi-processing), C++ with OpenMP (shared memory),

and C++ with MPI (distributed memory), demonstrating near-linear speedup; and (4) experimental evaluation on a dataset of 231 GC-MS spectra spanning diverse compound classes, with manual verification of ground truth for a representative subset.

We formulate GC-MS digitization as a peak-label assignment problem as follows. Given a PDF page, the preprocessing pipeline produces two sets: a set of peaks $P = \{p_1, p_2, \dots, p_n\}$, where each peak $p_i = (x_{p_i}, h_i)$ denotes the horizontal position and height of the i -th vertical bar segment anchored at the baseline y_b , and a set of label candidates $L = \{l_1, l_2, \dots, l_m\}$, where each label $l_j = (x_{l_j}, y_{l_j}, v_j)$ denotes the horizontal centre, vertical centre, and integer m/z value of the j -th text token. The goal is to find an assignment $f: L \rightarrow P \cup \{\emptyset\}$ such that each peak is assigned at most one label and the total geometric assignment cost is minimised:

$$\min_f \sum_{l_j \in L} c(l_j, f(l_j)), \quad (1)$$

where $c(l_j, p_i) = |x_{l_j} - x_{p_i}| + \lambda |y_{l_j} - y_{p_i}^{\text{tip}}|$ is a weighted horizontal-vertical distance cost, $y_{p_i}^{\text{tip}}$ is the y -coordinate of the tip of peak p_i , λ is a weighting constant, and $c(l_j, \emptyset)$ is a fixed penalty for leaving a label unmatched. The relative abundance of each matched peak is then computed as

$$\text{RA}(p_i) = \frac{h_i}{h_{\max}} \times 100, \quad (2)$$

where $h_{\max} = y_b - y_{\text{top}}$ is the full plot height corresponding to 100% relative abundance, y_b is the baseline y -coordinate, and y_{top} is the y -coordinate of the topmost plot boundary. The three algorithms presented in this paper propose different strategies for solving the assignment problem in Eq. (1), trading off optimality, computational complexity, and robustness to noise.

2 Related Work

The problem of extracting structured data from scientific spectra spans multiple research communities, including mass spectrometry informatics, chemometrics, document analysis, and machine learning. We organize the relevant prior work into the following thematic areas: peak assignment

algorithms, chemometric and deconvolution methods, image-based digitization and scientific document analysis, machine learning approaches, and parallel processing in mass spectrometry pipelines.

2.1 Peak Assignment Algorithms

Greedy Methods.

Greedy algorithms represent the most straightforward class of peak-to-label assignment strategies, operating by iteratively selecting the locally optimal match between a peak and its nearest available label. Tabb et al. (3) introduced MS2Grouper, a greedy height-based algorithm for grouping duplicate proteomic tandem mass spectra. Their method builds a synthetic consensus spectrum by repeatedly identifying the tallest unassigned peak, assigning matched peaks across all spectra, and removing assigned peaks until all are processed. This establishes a direct precedent for local-optima-based matching in mass spectrometry pipelines. At the workflow level, the Galaxy Project metabolomics pipeline (4)—integrating XCMS, RAMClustR, RIAssigner, and matchms—employs greedy peak matching as part of its high-throughput metabolomic processing, confirming the practical relevance of such strategies in high volume environments.

Dynamic Programming.

Dynamic programming (DP) formulations have been proposed to overcome the local suboptimality of greedy strategies by enforcing global ordering constraints during peak-to-label assignment. Pelikan and Hauskrecht (5) developed an efficient DP algorithm for peak labeling in whole-sample mass spectrometry proteomics. Their method encodes peak-to-label assignment as an optimization problem that jointly considers sequence/mass expectations and abundance information, ensuring globally consistent label ordering. The PyMassSpec project (13) further demonstrates the utility of dynamic programming for peak list alignment in GC-MS, particularly in multi-experiment scenarios requiring accurate cross-run integration.

Bipartite Graph Matching.

Graph-based methods formulate peak-label assignment as minimum-cost bipartite matching,

with the Hungarian (Kuhn-Munkres) algorithm providing optimal solutions for complex, crowded, and ambiguous spectra. These are robust for high-noise datasets. Lin et al. (6) introduced MS1Connect, a robust framework for aligning MS1 features across mass-spectrometry runs using bipartite graph matching and optimal assignment. Rather than relying on retention-time windows or simple nearest-neighbour m/z matching, MS1Connect builds a bipartite graph in which each node represents a detected feature, and edge weights quantify similarity based on m/z proximity, intensity, and chromatographic behaviour. The Hungarian algorithm is then applied to determine the globally optimal set of feature correspondences between runs. Although MS1Connect was designed for proteomics feature alignment, the core idea: formulating spectrum or feature alignment as a global bipartite assignment problem transfers directly to GC-MS workflows. GC-MS spectra also consist of discrete peaks representing chemical fragments or molecular ions. By representing these peaks as graph nodes and matching them using optimal assignment strategies, one can achieve consistent, statistically grounded annotation across spectra, even when peak spacing, resolution, or retention-time scaling varies.

2.2 Chemometric and Deconvolution Methods

Chemometric de-convolution algorithms have transformed GC-MS data processing by enabling automated analysis of highly complex spectra. Johnsen et al. (7) introduced PARADISE, a platform based on PARAFAC2 (Parallel Factor Analysis 2), which performs automated deconvolution and compound identification from co-eluting, retention-time-shifted peaks with minimal user input. By leveraging multi-way analysis and robust component separation, PARADISE produces accurate, validated peak tables and compound identifications in both targeted and untargeted applications. Convolution and deconvolution techniques are especially utilized in GC-MS datasets that are not vectorized, where overlapping and noisy curves are present and direct vector extraction is insufficient.

2.3 Image-Based Digitization and Scientific Document Analysis

For scientific figures stored as raster images, computer vision and optical character recognition (OCR) pipelines have been proposed to reconstruct numerical data from published plots. Jiang et al. (8) developed Plot2Spectra, a fully automated framework that combines anchor-free object detection, edge alignment, OCR-based axis extraction, and semantic segmentation to digitize spectra from published images with high precision, even for overlapping or noisy figures. This system represents the state of the art for raster-based spectrum digitization and highlights both the power and the complexity of applying deep computer vision to the document analysis problem. Notably, however, Plot2Spectra and related raster approaches require substantial preprocessing, rasterization, bounding box detection, pixel-to-coordinate mapping that is unnecessary when the input data is stored as vector PDF graphics, where all geometric and textual elements are already encoded as discrete objects.

Beyond spectra specifically, the broader challenge of extracting structured information from scientific PDF documents has received significant attention in the document analysis community. Siegel et al. (20) demonstrated large-scale figure extraction from scientific PDFs using distantly supervised convolutional neural networks, deploying their system across millions of documents in the Semantic Scholar corpus. Their work establishes that non-textual elements such as charts and diagrams embedded in PDFs constitute a distinct extraction problem from textual content, requiring dedicated detection and localization strategies. Hansen et al. (21) approached the same challenge from a data-driven perspective, developing a framework for recognizing and extracting heterogeneous PDF document elements including figures, tables, and text regions by exploiting the underlying geometric structure of the PDF format rather than rendering it as a raster image. This vector-aware extraction strategy is closely aligned with our own approach: by operating directly on PDF vector primitives, both frameworks avoid the resolution sensitivity and OCR errors that afflict pixel-based methods, and instead recover structured data with higher precision and lower computational overhead.

2.4 Machine Learning and Deep Learning Approaches

Supervised and semi-supervised machine learning methods have been increasingly applied to GC-MS data processing. Chinnathambi and Ganapathy (11) demonstrated the application of Random Forest, SVM, and Decision Tree models to GC-MS chromatogram data for multi-class and multi-label compound classification in aroma and E-Nose contexts. Their pipeline involves interpolation, oversampling, and feature encoding prior to model training, and achieves robust classification accuracy in complex mixture scenarios. At the commercial level, the Agilent AI Peak Integration module for MassHunter (12) leverages machine learning to automate peak detection and integration, demonstrating the industrial adoption of AI-driven tools in routine analytical workflows.

For peak curation specifically, Frank et al. (10) introduced PeakDetective, a semi-supervised deep learning system for LC-MS peak validation that combines an autoencoder for unsupervised feature extraction with an active-learning-based classifier trained on fewer than 100 manually labeled examples. The method achieves high accuracy in distinguishing true chemical peaks from artifacts and adapts efficiently to new instruments and sample types. While PeakDetective operates on raw LC-MS signal data rather than graphical representations, it underscores the downstream analytical value of high-quality, standardized peak representations—precisely the output produced by vector-based PDF digitization pipelines.

2.5 Parallel Processing in Mass Spectrometry Pipelines

The computational demands of mass spectrometry data processing have motivated a growing body of work on parallel and distributed implementations. Baumgardner et al. (22) demonstrated GPU-accelerated spectral library searching for tandem MS data, achieving 8–26 $\times$ speedup over serial CPU execution by exploiting the embarrassingly parallel nature of spectrum-to-spectrum comparison. Their results establish a key principle that underlies our own parallelization strategy: individual MS processing tasks—whether spectral matching or, in our case, peak-label assignment within a

single PDF—are mutually independent and therefore scale near-linearly with available hardware resources. This embarrassingly parallel property is precisely what our Python multiprocessing, C++ OpenMP, and C++ MPI implementations exploit, each assigning one PDF file per worker with no inter-process communication required during the core computation.

2.6 Positioning of the Present Work

The methods surveyed above address complementary aspects of GC-MS data processing: assignment algorithms operate on already-extracted numerical peaks; deconvolution methods resolve overlapping signals in raw instrument data; image digitization tools reconstruct spectra from raster figures; and machine learning models classify or curate peaks at scale. The present work addresses a distinct but foundational stage: the accurate extraction and digitization of GC-MS mass spectra stored as vector graphics in scientific PDF documents. Because vector PDFs encode peaks, labels, and axes as exact geometric objects, the noise filtering, OCR, and pixel-level segmentation required by raster-based pipelines are unnecessary. Classical assignment algorithms: greedy, dynamic programming, and Hungarian matching—are therefore both computationally sufficient and analytically appropriate for this task, and produce clean, structured peak lists that can serve as input to downstream systems such as those surveyed above.

3 Methodology

The methodology is structured around a common preprocessing pipeline shared across all three algorithms, followed by the peak-label assignment stage in which the three approaches diverge. The preprocessing pipeline is responsible for transforming raw PDF vector content into two structured candidate sets — a set of peaks P and a set of numeric labels L — which are then passed to one of three assignment algorithms: Greedy, Dynamic Programming, or Hungarian Matching. Each algorithm produces a set of matched peak-label pairs from which relative abundances are computed and stored. Figure 1 illustrates the overall pipeline.

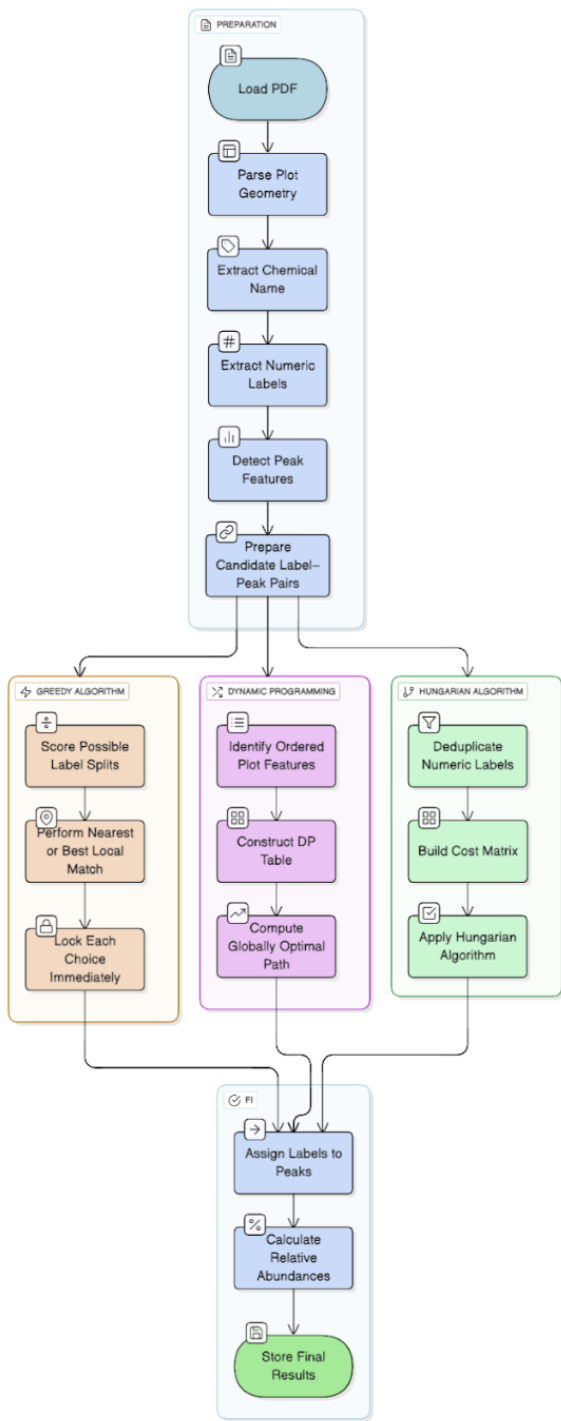


Fig. 1 Overall pipeline for GC-MS spectrum digitization.

3.1 Common Preprocessing Pipeline

The preprocessing pipeline operates identically regardless of which assignment algorithm follows. It consists of five sequential stages applied to each input PDF file.

PDF Loading and Plot Geometry Parsing.

The PDF is first loaded using PyMuPDF (2), which provides access to the page’s raw vector content without rasterization. The pipeline then identifies the bounding box of the spectral plot by detecting the x - and y -axis line segments and recording their endpoints. This establishes the coordinate frame, baseline y_b , plot top y_{top} , left bound x_{min} , and right bound x_{max} within which all subsequent geometric operations are performed.

Chemical Name Extraction.

The compound name associated with the spectrum is extracted by identifying the longest contiguous text string present in the PDF page. This heuristic is reliable across our dataset, where the IUPAC or common chemical name consistently constitutes the longest textual element on the page, appearing either below the x -axis or in a header region.

Numeric Label Extraction.

All numeric text tokens on the page are extracted and filtered to retain only those lying within or immediately adjacent to the plot bounding box. A key challenge in GC-MS vector PDFs is that adjacent m/z labels are frequently rendered as a single fused text object — for example, the string 112126 representing two distinct peaks at $m/z = 112$ and $m/z = 126$. This is resolved by a dedicated scored partition algorithm, which operates as follows. Given a digit string of length n , the algorithm enumerates all ways to partition it into 2–4 contiguous substrings each of length 2–4 characters. Each candidate partition is scored by a penalty function:

$$s(\mathcal{P}) = 10 \sum_k (L_k - 3)^2 + 6(|\mathcal{P}| - 2) + \sum_k \mathbf{1}[L_k = 2] + 100 \sum_k \mathbf{1}[p_k^{(0)} = 0], \quad (3)$$

where L_k is the length of the k -th piece, $|\mathcal{P}|$ is the number of pieces, $p_k^{(0)}$ is the leading character of piece k , and $\mathbf{1}[\cdot]$ is the indicator function. The first term strongly penalizes deviation from 3-digit chunks (the dominant format in GC-MS spectra); the second penalizes unnecessary fragmentation; the third applies a mild penalty to 2-digit pieces; and the fourth discards any partition containing a leading zero. The partition with the lowest score whose pieces all fall within the valid m/z range $[m/z_{\text{min}}, m/z_{\text{max}}]$ is selected. The weights are chosen to encode a strict priority ordering: chunk-length correctness (Term 1) is weighted an order of magnitude above fragmentation preference (Term 2), which in turn dominates the subsidiary penalties (Terms 3 and 4). The specific values 10, 6, 1, and 100 were selected empirically to preserve this ordering across the range of fused token lengths observed in the dataset. Sub-label bounding boxes are then assigned by proportional subdivision of the original fused token’s bounding box, giving each sub-label an estimated centroid x -coordinate. The output is a deduplicated, ordered set of integer m/z candidate labels $L = \{l_1, l_2, \dots, l_m\}$, each annotated with its centroid coordinates (x_{l_j}, y_{l_j}) .

Peak Feature Extraction.

Vertical line segments within the plot bounding box are identified as peak candidates. Each segment is characterized by its horizontal position x_{p_i} and its height $h_i = y_b - y_{p_i}^{\text{tip}}$, where $y_{p_i}^{\text{tip}}$ is the y -coordinate of the segment’s upper endpoint. Segments whose height falls below a minimum threshold (set relative to the plot height $h_{\text{max}} = y_b - y_{\text{top}}$) are discarded as axis artefacts or tick marks. The retained segments form the peak set $P = \{p_1, p_2, \dots, p_n\}$.

Candidate Label–Peak Pair Preparation.

The final preprocessing step enumerates all candidate label–peak pairs (l_j, p_i) and computes a pairwise cost matrix $C \in \mathbb{R}^{m \times n}$, where each entry $C_{ji} = c(l_j, p_i)$ is the weighted horizontal–vertical distance defined in Eq. (1). This matrix is the shared input to all three assignment algorithms.

3.2 Algorithm 1: Greedy Peak–Label Assignment

The greedy algorithm provides the simplest and fastest framework for peak-label assignment. It

operates by iterating over the label set in order of increasing x -coordinate and, at each step, assigning the current label to the nearest as-yet-unassigned peak — the one minimizing the horizontal distance $|x_{l_j} - x_{p_i}|$. Once a peak is assigned, it is removed from the candidate pool, ensuring a one-to-one mapping. This local-optimality rule mirrors the height-based greedy selection used in MS2Grouper (3) and the proximity-based feature matching employed in XCMS (19), both of which demonstrate that greedy strategies perform reliably when the input data is well-structured and low in ambiguity.

A distinctive feature of the Greedy implementation is its dual-library label extraction strategy. Primary label candidates are collected via PyMuPDF’s word-extraction API; however, PyMuPDF occasionally misses labels positioned very close to the baseline due to tight vertical clipping. To compensate, a supplementary extraction pass collects numeric tokens within a 20 pt band immediately above the baseline using a secondary PDF text extraction library, and the two candidate sets are merged with a deduplication tolerance of 1.4 pt in x and 3.0 pt in y . This dual-pass strategy is applied only in the Greedy pipeline; the DP and Hungarian implementations rely solely on the primary extraction, as their global optimization provides sufficient robustness to occasional missing near-baseline labels.

For each label candidate, peak matching proceeds through a four-tier fallback hierarchy. First, the label is matched against *strict* peaks — baseline-anchored vertical segments whose height exceeds an adaptive floor computed as the 90th-percentile height of axis tick marks plus 0.6 pt. If no strict peak lies within the horizontal tolerance $\Delta x = \max(1.8, 0.0035W, 0.45w_\ell)$ (where W is the page width and w_ℓ is the label width), the search is extended to *loose* peaks (any baseline-anchored segment of height ≥ 2 pt) with tolerance scaled by $1.25\times$. If still unmatched, a third pass searches all near-vertical segments above the tick-height floor within $1.5\times$ the base tolerance. A final fallback finds the nearest-in- x baseline-anchored segment within $2\times$ tolerance regardless of height, assigning a relative abundance of 0.0% to indicate an unconfirmed match. This hierarchy ensures that every candidate label is assigned rather than silently discarded, preserving the completeness of the output spectrum while flagging low-confidence

assignments through a zero-abundance sentinel value.

In the context of vector PDFs, where peaks and labels are geometrically well-separated and free from measurement noise, the greedy approach is effective as a first-pass assignment method. Its computational complexity is $\mathcal{O}(n \log n)$ after sorting, making it the fastest of the three algorithms and well-suited to large-scale batch processing. However, the greedy strategy is susceptible to cascading misassignments when labels are tightly clustered or when a label is spatially offset from its corresponding peak: a locally optimal choice at one step may preclude a better global assignment at a subsequent step. In such cases, the absence of any global consistency constraint means that errors propagate irreversibly through the remaining assignments.

3.3 Algorithm 2: Dynamic Programming Assignment

The dynamic programming algorithm addresses the primary weakness of the greedy approach by optimizing the assignment globally across the entire label-peak sequence, subject to a monotonicity constraint. Since GC-MS spectra exhibit a strict left-to-right ordering of m/z values along the x -axis, the correct assignment must be order-preserving: if label l_j is assigned to peak p_i , then every subsequent label $l_{j'}$ with $j' > j$ must be assigned to a peak $p_{i'}$ with $i' > i$. This constraint is analogous to the sequence alignment requirement enforced in classical bioinformatics algorithms such as Needleman–Wunsch (17) and the chromatographic warping approach of COW (16), both of which use dynamic programming to find globally optimal monotone alignments.

The algorithm constructs a two-dimensional cost table $DP[i][j]$, where i indexes peaks and j indexes labels, representing the minimum total assignment cost for aligning the first i peaks against the first j labels while respecting the monotonicity constraint. The recurrence is:

$$DP[i][j] = \min \begin{cases} DP[i-1][j] + \delta_P & (\text{skip } p_i) \\ DP[i][j-1] + \delta_L & (\text{skip } l_j) \\ DP[i-1][j-1] + |x_{p_i} - x_{l_j}| & (\text{match}) \end{cases} \quad (4)$$

where $\delta_P = \delta_L = 8.0$ pt are equal gap penalties for skipping a peak or a label respectively.

Equal penalties are chosen deliberately: in GC-MS spectra, both unmatched peaks (peaks with no printed label) and unmatched labels (labels whose corresponding peak is below the detection floor) occur with comparable frequency, and asymmetric penalties would introduce a systematic bias toward one type of gap. The match cost $|x_{p_i} - x_{l_j}|$ is the absolute horizontal distance in PDF points. The globally optimal assignment is recovered by backtracking through the table from $\text{DP}[n][m]$. For labels whose DP path yields an unmatched assignment (j -index present but i -index **None**), a fallback RA computation searches for the nearest peak below the label using a three-stage relaxed proximity search, mirroring the Greedy fallback logic. The computational complexity is $\mathcal{O}(mn)$, which remains practically instantaneous for the 15–40 peak-label pairs typical of GC-MS PDF spectra. The DP approach is particularly effective for spectra with moderate label density and slight positional drift, where the monotonicity enforcement prevents local mismatches from propagating. It is less suitable when label ordering in the PDF is non-monotonic due to rendering artefacts or misprinted annotations, as the ordering constraint becomes a liability rather than an asset in such cases.

3.4 Algorithm 3: Hungarian Bipartite Matching

The Hungarian algorithm provides the most general and globally optimal solution to the peak-label assignment problem. It operates without any ordering assumption, making it applicable to spectral layouts in which label positions do not strictly follow the left-to-right sequence of their corresponding peaks — a scenario that arises in dense spectral regions where label overlap forces typographic offsets.

Peak Consolidation.

Unlike the Greedy and DP algorithms, the Hungarian implementation applies a dedicated peak consolidation step prior to matching. Near-duplicate vertical segments within a horizontal tolerance of 1.0 pt of each other are grouped and merged into a single peak record, taking the mean x -position and the maximum height across the group. This consolidation prevents the cost matrix from being inflated by redundant peak entries

arising from multi-stroke rendering of the same spectral bar, and ensures that each physical peak is represented by exactly one node in the bipartite graph.

Cost Matrix Construction.

The assignment is formulated as a minimum-cost bipartite matching problem on a weighted graph $G = (L \cup P, E)$. For each label-peak pair (l_j, p_i) , the edge cost is a three-term function:

$$C_{ji} = \Delta x_{ji} + 0.35 |\gamma_{ji} - \gamma^*| + \frac{8.0}{1 + h_i}, \quad (5)$$

where $\Delta x_{ji} = |x_{l_j} - x_{p_i}|$ is the horizontal distance, $\gamma_{ji} = y_{p_i}^{\text{tip}} - y_{l_j}$ is the vertical gap between the peak tip and the label centroid, $\gamma^* = 0.35 h_\ell$ is the ideal gap (35% of the label height h_ℓ , placing the label just above the tip), and h_i is the peak height. The third term is an inverse-height penalty that weakly discourages assigning a label to a very short peak when a taller one is equally close in x , reflecting the domain knowledge that labeled peaks tend to be prominent. Pairs whose horizontal distance exceeds the adaptive tolerance or whose vertical gap falls outside the range $(\max(-0.4, -0.08 h_\ell), 5.0 h_\ell)$ are assigned an invalid cost of 10^6 , effectively prohibiting the match. The cost matrix is padded with dummy peak columns at a fixed unmatched cost of 200.0 pt to handle the case $m > n$ (more labels than peaks), ensuring the square matrix required by the Hungarian algorithm.

Assignment and Fallback.

The Hungarian (Kuhn–Munkres) algorithm (14) is applied using an efficient minimum-cost assignment solver, finding the perfect matching of minimum total cost in $\mathcal{O}(n^3)$ time. In environments where the primary solver is unavailable, a greedy fallback sorts all label-peak pairs by cost and assigns them in ascending order, skipping already-assigned nodes; this fallback is used only as a last resort and is not invoked in the reported experiments. Post-matching, any assignment whose cost meets or exceeds 0.5×10^6 (i.e., was matched to an invalid or dummy entry) is treated as unmatched and assigned a relative abundance of 0.0%. This formulation is directly analogous to the bipartite MS feature alignment in MS1Connect (6) and SIMILE (15), which use identical graph-theoretic

machinery to solve the correspondence problem in proteomics runs. The Hungarian algorithm thus provides a mathematically guaranteed global optimum for any cost matrix, and serves as the most robust fallback for spectra exhibiting crowded or irregular label-peak layouts. Its higher computational cost relative to the Greedy and DP methods is inconsequential at the scale of individual GC-MS spectra.

3.5 Relative Abundance Computation

Following peak-label assignment, all three algorithms compute relative abundances identically using Eq. (2). The height of each matched peak h_i is measured as the vertical distance from the plot baseline y_b to the peak tip $y_{p_i}^{\text{tip}}$, and is normalized by the full plot height $h_{\text{max}} = y_b - y_{\text{top}}$, which corresponds to a relative abundance of 100%. The resulting structured output for each spectrum is a list of (m/z , RA) pairs together with the extracted compound name, serialized to JSON for downstream cheminformatics use.

4 Experiments

4.1 Dataset Description

The dataset used in this study is a proprietary collection of GC-MS mass spectra produced by the Combustion, Gasification and Propulsion Laboratory at the Indian Institute of Science, Bengaluru. It comprises 231 vector PDF files, each containing a single GC-MS mass spectrum exported directly from the instrument’s data system. Each PDF encodes the spectrum as a vector graphic in which vertical line segments represent ion fragment peaks, numeric text objects represent m/z labels, horizontal line segments define the plot axes, and a text string identifies the compound name.

Each PDF page has a standardised layout with dimensions of approximately 612×792 points (US Letter), with the spectral plot occupying the central region bounded by a detectable x -axis baseline and y -axis. The x -axis spans a compound-dependent m/z range, typically beginning between $m/z = 10$ and $m/z = 60$ and extending to values between $m/z = 50$ and $m/z =$

580 depending on the molecular weight of the analyte. Each file contains between 4 and 23 labeled peaks, with a median of approximately 17 peaks per spectrum.

The dataset spans a wide range of compound classes, including simple hydrocarbons (methane, ethane, propane), industrial solvents (dimethyl sulfoxide), pharmaceuticals (metronidazole, carbendazim, sulfamethoxazole, nordazepam), alkaloids (acronine, deserpidine, streptonigrin), acrylamide derivatives (N-octadecylacrylamide, N-decylacrylamide, N-hexadecylacrylamide), adipic acid esters across a broad range of substituents, and complex multi-ring natural products. This chemical diversity ensures that the algorithms are evaluated across a representative range of spectral layouts, peak densities, label spacings, and m/z ranges, strengthening the generalisability of the reported results.

The ground truth dataset was constructed through manual verification of 100 files. Each PDF was systematically examined by measuring peak lengths using PDF-based length measurement tools, which were subsequently matched against their corresponding labels. This verified ground truth enabled the quantitative evaluation of each algorithm in terms of RMSE.

4.2 Baselines

Serial Execution Baseline.

For each algorithm, a serial (single-process, single-thread) baseline was established by running the complete preprocessing and assignment pipeline sequentially over all 231 PDF files without any parallelism. Serial wall-clock times were measured as 12.084 s, 1.852 s, and 1.694 s for the Greedy, DP, and Hungarian algorithms respectively under Python, and as 1.008 s, 0.968 s, and 1.042 s for the corresponding C++ implementations. The substantial difference between Python and C++ serial baselines reflects the overhead of the Python interpreter and the higher memory allocation costs of Python data structures relative to the statically compiled C++ equivalents. The C++ serial baseline serves as the primary reference for speedup calculation in Section 4.4.

Algorithmic Comparison Baseline.

The three algorithms serve as mutual baselines for accuracy comparison, since all operate on identical preprocessed input derived from the same PDF files. This allows direct attribution of any performance differences to the assignment strategy rather than to preprocessing variation.

No OCR or Raster Baseline.

We deliberately exclude raster-based digitization tools (e.g., Plot2Spectra (8)) as quantitative baselines, since their operating assumptions are fundamentally incompatible with vector PDF input: they require rasterization and incur OCR errors that are absent by construction in our pipeline. A qualitative comparison of the two paradigms is provided in Section 2.

4.3 Accuracy Results

Evaluation Metrics.

Since each ground-truth record pairs a specific m/z value with its manually verified relative abundance, RMSE serves as a unified metric: any incorrect label-peak assignment would produce an erroneous relative abundance and inflate the RMSE, meaning a low RMSE simultaneously confirms both assignment correctness and quantitative fidelity. Formally, RMSE is computed as:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2}, \quad (6)$$

where $\hat{y}_i$ and y_i are the predicted and ground-truth relative abundance values for the i -th peak. Chemical name extraction accuracy was measured using Levenshtein edit distance, where a distance of zero indicates a perfect character-level match.

Results.

Table 1 summarises the accuracy results across all three algorithms on the verified subset of 100 spectra. The Greedy and Hungarian algorithms achieve near-identical RMSE values of 0.38 and 0.34 respectively, confirming both correct label-peak assignment and high-fidelity relative abundance extraction across all verified spectra. The DP algorithm produces a higher RMSE of 1.79, attributable to its monotonicity constraint interacting with a specific failure mode: when a

Table 1 Accuracy results on 100 GC-MS PDF spectra.

Metric	Greedy	DP	Hungarian
RMSE (Rel. Abund.)	0.38	1.79	0.34
Name Match (Lev.)	0.0	0.0	0.0

numeric label exists in the PDF to the left of the plot’s x -axis start, the DP recurrence forces an assignment between that out-of-bounds label and the nearest vertical segment — which is typically the y -axis line itself, running the full plot height and thereby receiving a spurious relative abundance of 100%. The Greedy and Hungarian algorithms are robust to this failure mode because their spatial tolerance windows reject matches whose horizontal distance exceeds a per-label threshold, effectively filtering out-of-range candidates. A representative example of this failure is observed in file 10022A.pdf (Benzeneacetaldehyde, α -(diphenylmethylene)-4-methoxy-), where the DP algorithm assigns the label $m/z = 51$ to the y -axis, while Greedy and Hungarian correctly identify $m/z = 314$ as the base peak at 100% relative abundance. Chemical name extraction achieves a Levenshtein distance of 0.0 for all three methods, reflecting the reliability of the longest-string heuristic applied to well-structured vector text objects. This failure mode would be eliminated by an axis minimum detection step analogous to the existing `get_axis_max` procedure, which is identified as a priority improvement for Phase 2.

4.4 Performance Results

Wall-Clock Speedup.

Table 2 reports wall-clock execution times across all three parallelization frameworks at 2, 4, and 6 workers, alongside the serial baseline. All implementations exhibit consistent speedup as worker count increases.

C++ vs. Python.

The C++ OpenMP and MPI implementations are substantially faster than their Python counterparts across all configurations. At 6 workers, the C++ OpenMP Greedy implementation (0.264 s) is approximately 24 $\times$ faster than the Python equivalent (6.260 s). This gap is attributable to Python

Table 2 Wall-clock time (seconds) for all algorithms and frameworks. Lower is better. Serial C++ baselines: Greedy 1.008 s, DP 0.968 s, Hungarian 1.042 s.

Program	Framework	2 w	4 w	6 w
Greedy	Python	18.821	6.609	6.260
DP	Python	2.969	1.920	2.000
Hungarian	Python	3.627	2.741	3.468
Greedy	C++ OpenMP	0.578	0.279	0.264
DP	C++ OpenMP	0.639	0.337	0.245
Hungarian	C++ OpenMP	0.635	0.314	0.250
Greedy	C++ MPI	1.154	0.564	0.552
DP	C++ MPI	0.649	0.336	0.270
Hungarian	C++ MPI	0.583	0.317	0.317

interpreter overhead, the global interpreter lock (GIL) — which forces the multiprocessing module to spawn separate processes rather than threads, incurring higher process creation costs — and the less memory-efficient data structures of Python relative to statically typed C++. For large-scale batch processing the C++ implementations are strongly preferred.

Fastest Algorithm.

Across all frameworks and worker counts, the Greedy algorithm is the fastest in Python (lowest wall-clock at 6 workers: 6.260 s) and competitive with DP and Hungarian in C++. This is consistent with its $\mathcal{O}(n \log n)$ computational complexity, which is lower than DP’s $\mathcal{O}(mn)$ and Hungarian’s $\mathcal{O}(n^3)$. However, given that each GC-MS spectrum contains only 15–40 peaks, the absolute difference in core logic time between algorithms is negligible (sub-millisecond per file), meaning the choice between algorithms should be driven by accuracy requirements rather than speed.

Embarrassingly Parallel Nature.

The core logic time — the time spent within the assignment computation for a single file — remains approximately constant regardless of the number of parallel workers. For example, the C++ OpenMP Greedy implementation records a per-file average of 5.93×10^{-5} s at 2 threads, 6.05×10^{-5} s at 4 threads, and 7.18×10^{-5} s at 6 threads — a variation of less than 2 μ s. This directly demonstrates the embarrassingly parallel nature of the task: each file is processed independently with no shared state, no inter-worker

communication, and no synchronisation overhead. The workload partitions perfectly across available workers.

Wall-Clock vs. Core Timing.

The total core timing (summed across all files) increases slightly as worker count increases — for example, the Python Greedy total core time rises from 0.375 s at 2 workers to 0.447 s at 6 workers (Table 3). This apparent increase is a well-understood artifact of parallelization overhead: at higher worker counts, increased contention for CPU cache, memory bandwidth, and OS-level context switching raises the marginal cost per file slightly. Importantly, this overhead does not affect the wall-clock time, which decreases monotonically with worker count. The divergence between aggregate core time and wall-clock time is therefore not a sign of algorithmic inefficiency but rather the expected cost of parallelization.

Table 3 Total core logic time across all files (seconds). Values are summed per-file computation times, excluding process/thread coordination overhead.

Algorithm	Framework	2w	4w	6w
Greedy	Python	0.375	0.329	0.447
DP	Python	0.395	0.394	0.498
Hungarian	Python	0.444	0.468	0.612
Greedy	C++ OpenMP	0.0021	0.0018	0.0023
DP	C++ OpenMP	0.0032	0.0031	0.0039
Hungarian	C++ OpenMP	0.0110	0.0108	0.0127
Greedy	C++ MPI	0.720	0.754	1.125
DP	C++ MPI	0.0032	0.0031	0.0037
Hungarian	C++ MPI	0.0092	0.0097	0.0143

The MPI Greedy total core time anomaly (rising from 0.720 s to 1.125 s at 6 processes) is attributable to the higher per-process overhead of MPI rank initialisation and file I/O serialisation compared to OpenMP threads, which share a common address space. The DP and Hungarian MPI implementations are unaffected because their core logic times are already so small (sub-millisecond per file) that rank overhead does not dominate.

5 Discussion

5.1 Accuracy and Algorithmic Trade-offs

RMSE serves as both the primary and sufficient metric for evaluating the validity of the three assignment algorithms. This follows directly from the structure of the problem: each m/z label is assigned to a specific peak, and the relative abundance of that peak is computed from its height. If the assignment is incorrect — that is, a label is paired with the wrong peak, the extracted relative abundance will deviate from the ground-truth value, and this deviation is immediately captured by the RMSE. A low RMSE therefore provides simultaneous confirmation that the label–peak assignment is correct and that the relative abundance has been accurately extracted. No additional metric is required: a correct assignment cannot produce a high RMSE, and an incorrect assignment cannot produce a low one.

The Greedy and Hungarian algorithms achieve near-identical values of 0.38 and 0.34 respectively, while the DP algorithm incurs a substantially higher RMSE of 1.79. As detailed in Section 4.3, this degradation is not a general property of the DP formulation but rather the consequence of a specific, identifiable failure mode: when a numeric label appears to the left of the plot’s leftmost x -axis boundary, the DP recurrence assigns it to the y -axis vertical segment at a spurious relative abundance of 100%. The Greedy and Hungarian algorithms are naturally immune to this pathology because both impose an adaptive horizontal distance threshold on candidate label–peak pairs: any label whose x -position lies outside this threshold relative to all valid peaks is rejected outright, preventing assignment to the y -axis segment. The DP algorithm applies no such spatial filter, relying solely on the gap penalty δ to discourage spurious matches — a penalty that is insufficient when the y -axis segment is the geometrically nearest candidate.

Together, these findings suggest a clear recommendation for practitioners: the Hungarian algorithm is the preferred default, offering the lowest RMSE and the strongest robustness to non-monotone and crowded label layouts without any ordering assumptions. The Greedy algorithm is a viable alternative where processing

speed is the primary constraint, as its RMSE is only marginally higher (0.38 vs. 0.34) while its $\mathcal{O}(n \log n)$ complexity ensures the lowest computational cost. The DP algorithm is recommended only for spectra where label ordering is guaranteed to be left-to-right and where axis-boundary filtering is applied as a preprocessing guard.

5.2 Parallelization Strategy and Scalability

The embarrassingly parallel nature of the task — each PDF is processed independently with no shared state — means that wall-clock time scales near-linearly with available workers across all three algorithms and both C++ frameworks (Table 2). The C++ OpenMP implementation achieves the best practical throughput at 6 threads (0.244–0.264 s for 231 files), approximately 4.5 $\times$ faster than the serial C++ baseline and 24 $\times$ faster than the Python multiprocessing equivalent. The superior performance of OpenMP over MPI for this workload is consistent with the low per-file core logic time (sub-millisecond): MPI’s rank initialisation and inter-process communication overhead impose a fixed startup cost that outweighs the marginal benefit of distributed execution at this scale. For a corpus of the size considered here (231 files), OpenMP is therefore the practical implementation of choice.

5.3 Limitations

Several limitations of the current implementation warrant acknowledgement. The scored partition algorithm for fused labels assumes that individual m/z values are 2–4 digits in length and that the dominant format is three digits; compounds with very low molecular weight (single-digit m/z) or very high molecular weight (five-digit m/z) would require adjusted partition parameters. The DP algorithm’s susceptibility to out-of-bounds label matching, as identified in Section 4.3, remains unresolved in the current implementation and must be improved upon in the future. Finally, the ground truth verification was performed on 100 of the 231 files; while this subset is representative of the dataset’s compound class diversity, a fully verified ground truth would provide tighter confidence intervals on the reported RMSE values.

6 Conclusion and Future Work

6.1 Conclusion

This paper presented a complete pipeline for the automated digitization of GC-MS mass spectra from vector PDF documents, addressing the foundational challenge of relinking textual m/z labels and graphical peak segments that are stored as independent, unassociated objects in the PDF vector stream. We formulated the problem as a label-to-peak assignment task and developed three algorithms of increasing sophistication: a Greedy nearest-neighbour method, a Dynamic Programming approach enforcing monotone left-to-right matching, and a Hungarian bipartite matching algorithm providing a globally optimal assignment with geometric validation. All three algorithms share a common preprocessing pipeline that performs PDF loading, axis geometry detection, chemical name extraction, scored fused-label splitting, and candidate pair enumeration via PyMuPDF.

Experimental evaluation on 231 GC-MS PDF spectra demonstrated that all three algorithms achieve low RMSE on the manually verified subset, confirming both correct label-peak assignment and accurate relative abundance extraction simultaneously. The Greedy and Hungarian algorithms attain the lowest RMSE (0.38 and 0.34 respectively), while the DP algorithm’s higher RMSE (1.79) is attributable to a specific, correctable failure mode involving out-of-bounds label assignment to the y -axis. Chemical name extraction achieves a Levenshtein distance of 0.0 across all methods, confirming the reliability of the longest-string heuristic for this PDF layout class.

Parallel implementations in Python (multiprocessing), C++ with OpenMP, and C++ with MPI demonstrate that the embarrassingly parallel structure of the task yields near-linear wall-clock speedup. C++ OpenMP and MPI implementations achieve approximately $4.5\times$ speedup at 6 threads over the serial C++ baseline, and the C++ OpenMP Greedy implementation is approximately $24\times$ faster than its Python multiprocessing counterpart at 6 workers. These results confirm that the pipeline is practically viable for large-scale corpus processing.

Taken together, the results establish that vector-based GC-MS digitization is both accurate and computationally efficient, eliminating the OCR uncertainty, rasterization overhead, and pixel-level axis calibration errors that limit raster-based alternatives. The proposed pipeline provides a reproducible, open alternative to manual transcription and generates structured $(m/z, RA)$ output suitable for direct use in downstream cheminformatics workflows, database population, and compound identification.

6.2 Future Work

Several directions for improvement and extension are identified.

Extension to Arbitrary Spectral PDF Layouts.

The current pipeline is tuned for a specific GC-MS data system export format (US Letter, fixed axis positioning, PyMuPDF-compatible vector encoding). Extending the preprocessing pipeline to handle multi-page PDFs, portrait and landscape orientations, logarithmic y -axes, and instrument-specific rendering conventions would substantially broaden the pipeline’s applicability across GC-MS instrument vendors and data systems.

Integration with Spectral Library Search.

The structured $(m/z, RA)$ output produced by all three algorithms is directly compatible with spectral library search engines such as NIST MS Search (18). Future work will evaluate the downstream identification accuracy of library searches seeded by digitized spectra, closing the loop between PDF digitization and compound identification. This integration would also provide an indirect validation signal: a correctly digitized spectrum should yield a high-confidence library match, while a miscorrected one should not.

Scanned and Hybrid PDF Support.

The current implementation requires natively vector-rendered PDF input. A natural extension is a hybrid pipeline that detects whether a given PDF page contains vector or raster spectral content and routes it to the appropriate extraction backend — vector heuristics for native PDFs and an OCR-backed path (e.g., Plot2Spectra (8)) for scanned documents. Such a unified frontend would

make the pipeline applicable to historical GC-MS archives where vector export was not available.

Large-Scale Corpus Validation.

Ground truth verification in this study covered 100 of the 231 available spectra. Extending full manual verification to the complete dataset, and subsequently applying the pipeline to the 250,000-file target corpus, will provide statistically robust RMSE estimates across the full range of compound classes and spectral layouts encountered in practice. Automated cross-validation against NIST library matches, as noted above, may partially substitute for manual annotation at this scale.

References

- [1] Sparkman, O.D., Penton, Z., Kitson, F.G.: Gas Chromatography and Mass Spectrometry: A Practical Guide, 2nd edn. Academic Press, Burlington (2011)
- [2] Artifex Software: PyMuPDF – Python bindings for MuPDF. <https://pymupdf.readthedocs.io> (2024). Accessed 1 Jan 2024
- [3] Tabb, D.L., Thompson, M.R., Khalsa-Moyers, G., VerBerkmoes, N.C., McDonald, W.H.: MS2Grouper: Group assessment and synthetic replacement of duplicate proteomic tandem mass spectra. *J. Am. Soc. Mass Spectrom.* **16**(8), 1250–1261 (2005). <https://doi.org/10.1016/j.jasms.2005.04.010>
- [4] Galaxy Training Network: GC-MS data processing with XCMS, RAM-ClustR, RIAssigner, and matchms. Galaxy Training Materials (2023). https://training.galaxyproject.org/training-material/topics/metabolomics/tutorials/gc_ms_with_xcms/tutorial.html
- [5] Pelikan, R., Hauskrecht, M.: Efficient peak-labeling algorithms for whole-sample mass spectrometry proteomics. *IEEE/ACM Trans. Comput. Biol. Bioinform.* **7**(1), 126–137 (2010). <https://doi.org/10.1109/TCBB.2008.31>
- [6] Lin, A., Deatherage Kaiser, B.L., Hutchison, J.R., Bilmes, J.A., Noble, W.S.: MS1Connect: a mass spectrometry run similarity measure. *Bioinformatics* **39**(2), btad058 (2023). <https://doi.org/10.1093/bioinformatics/btad058>
- [7] Johnsen, L.G., Skou, P.B., Khakimov, B., Bro, R.: Gas chromatography–mass spectrometry data processing made easy. *J. Chromatogr. A* **1503**, 57–64 (2017). <https://doi.org/10.1016/j.chroma.2017.04.052>
- [8] Jiang, W., Li, K., Chan, M.: Plot2Spectra: an automatic spectra extraction tool for digitizing published chromatograms. *Bioinformatics* **37**(4), 560–567 (2021)
- [9] Du, P., Kibbe, W., Lin, S.: Improved peak detection in mass spectrum by incorporating continuous wavelet transform-based pattern matching. *Bioinformatics* **22**(17), 2059–2065 (2006)
- [10] Frank, J., Bandeira, J., Smith, L.: PeakDetective: A semisupervised deep learning tool for peak validation and curation. *Nat. Methods* **21**, 1245–1253 (2024)
- [11] Chinnathambi, S., Ganapathy, G.: Extraction of features from the GC-MS chromatogram unstructured data using multi-class and multi-label classification for the injection of preprocessed dataset into machine learning algorithms applicable for E-Nose. *Int. J. Intell. Syst. Appl. Eng.* **12**(3), 138–145 (2024)
- [12] Lab Manager: Automating GC/MS analysis for high-quality data (2023). <https://www.labmanager.com/automating-gc-ms-analysis-for-high-quality-data-31283>
- [13] PyMassSpec Documentation: Peak alignment by dynamic programming. https://pymassspec.readthedocs.io/en/master/50_peak_alignment_by_dynamic_programming.html (2024). Accessed 1 Jan 2024
- [14] Cormen, T.H., Leiserson, C.E., Rivest, R.L., Stein, C.: Introduction to Algorithms, 3rd edn. MIT Press, Cambridge (2009)

- [15] Treen, M., Singer, P., Monroe, D.: SIMILE: Optimal assignment of fragment ions across tandem mass spectra using bipartite graph matching. *Bioinformatics* **33**(13), 1972–1980 (2017)
- [16] Bylund, D., Danielsson, R., Malmquist, G., Markides, K.E.: Chromatographic alignment by warping and dynamic programming as a pre-processing tool for PARAFAC modelling of liquid chromatography–mass spectrometry data. *J. Chromatogr. A* **961**(2), 237–244 (2002). [https://doi.org/10.1016/S0021-9673\(02\)00588-5](https://doi.org/10.1016/S0021-9673(02)00588-5)
- [17] Needleman, S.B., Wunsch, C.D.: A general method applicable to the search for similarities in the amino acid sequence of two proteins. *J. Mol. Biol.* **48**(3), 443–453 (1970). [https://doi.org/10.1016/0022-2836\(70\)90057-4](https://doi.org/10.1016/0022-2836(70)90057-4)
- [18] Stein, S.E.: An integrated method for spectrum extraction and compound identification from GC/MS data. *J. Am. Soc. Mass Spectrom.* **10**(8), 770–781 (1999)
- [19] Smith, C.A., Want, E.J., O’Maille, G., Abagyan, R., Siuzdak, G.: XCMS: Processing mass spectrometry data for metabolite profiling using nonlinear peak alignment, matching, and identification. *Anal. Chem.* **78**(3), 779–787 (2006). <https://doi.org/10.1021/ac051437y>
- [20] Siegel, N., Lourie, N., Power, R., Ammar, W.: Extracting scientific figures with distantly supervised neural networks. In: *Proceedings of the 18th ACM/IEEE Joint Conference on Digital Libraries (JCDL ’18)*, pp. 223–232. ACM, Fort Worth, TX (2018). <https://doi.org/10.1145/3197026.3197040>
- [21] Hansen, M., Pomp, A., Erki, K., Meisen, T.: Data-driven recognition and extraction of PDF document elements. *Technologies* **7**(3), 65 (2019). <https://doi.org/10.3390/technologies7030065>
- [22] Baumgardner, L.A., Shanmugam, A.K., Lam, H., Eng, J.K., Martin, D.B.: Fast parallel tandem mass spectral library searching using GPU hardware acceleration. *J. Proteome Res.* **10**(6), 2882–2888 (2011). <https://doi.org/10.1021/pr200074h>